UNIVERSITY & STAFF ENGINEERING INSTRUCTOR MANUAL | DUAL-TRACK EDITION (agy & claude)

# Specification-Driven Development (SDD) with Conductor & OpenSpec

Lecturer Delivery Playbook, Slide-by-Slide Pacing Guide, Live Projector Demos, Complete Lab 01–06 Answer Keys, 3-Rung TA Hint Ladders, and Capstone Grading Rubrics

**Author:** Alejandro Kaspar — AI Forward Deployed Engineer **License:** CC BY 4.0 (Text/Diagrams) & Apache 2.0 (Code/TeX)

**Companion Assets:** Coursebook_SDD_Conductor_OpenSpec.pdf | 36-Slide Deck | 6 Self-Diagnosing Python Labs (TaskFlow)

## Contents

**1 Course Architecture, Cognitive Load & Turnkey Syllabi** **2**

1.1 Pedagogical Philosophy: From Ephemeral Prompting to Durable Specifications . . . . . 2

1.2 Sweller's Cognitive Load Theory and the Unified TaskFlow Progression . . . . . 2

1.3 Three Turnkey Pacing Syllabi (University, Bootcamp, and Staff Workshop) . . . . . 3

1.4 Dual-Track CLI Support (main vs. claude) and Deterministic Autograding . . . . . 4

**2 Lecture Delivery Playbook: Modules 1 to 3 (Slides 01–16)** **5**

2.1 Module 01 Delivery (Slides 01–06): Foundations, Spec Triangle & Dual-Engine Wiring . . . . . 5

2.2 Module 02 Delivery (Slides 07–11): Lab 01 — Greenfield Proposals to Specs (TodoList) . . . . . 6

2.3 Module 03 Delivery (Slides 12–16): Lab 02 — Brownfield Spec Recovery (TaskOrganizer) . . . . 7

**3 Lecture Delivery Playbook: Modules 4 to 7 (Slides 17–36)** **8**

3.1 Module 04 Delivery (Slides 17–21): Lab 03 — From Specs to TDD Code (HabitTracker) . . . . 9

3.2 Module 05 Delivery (Slides 22–26): Lab 04 — Drift Detection & 5-Step Scorecard Refactoring (DailyPlanner) . . . . . 9

3.3 Module 06 Delivery (Slides 27–31): Lab 05 — SDD Code Review & Stacked PRs (NotesBoard) . 10

3.4 Module 07 Delivery (Slides 32–36): Lab 06 — Capstone SSOT & Clean-Room Rebuild Test (TaskFlowBoard) . . . . . 11

**4 Lab 01–06 Facilitation, Answer Keys & 3-Rung TA Hint Ladders** **12**

4.1 The 3-Rung Socratic TA Facilitation Protocol . . . . . 12

4.2 Complete Answer Keys & 3-Rung Hint Ladders for Labs 01–03 . . . . . 12
4.3 Complete Answer Keys & 3-Rung Hint Ladders for Labs 04–06 . . . . . 13

**5 Assessments, Capstone Grading Rubrics & Primary Literature 15**
5.1 100-Point Capstone Specification & Clean-Room Rebuild Grading Rubric . . . . . 15
5.2 Six Open-Ended Architectural Design Exam Problems & Model Grading Keys . . . . . 17
5.3 Instructor's Annotated Primary-Source Literature Guide . . . . . 18

# 1 Course Architecture, Cognitive Load & Turnkey Syllabi

## 1.1 Pedagogical Philosophy: From Ephemeral Prompting to Durable Specifications

**Specification-Driven Development (SDD)** shifts software engineering from ephemeral, conversational prompt iteration to a disciplined engineering lifecycle in which versioned, testable Markdown specifications serve as the executable contract between humans and coding agents [2, 3]. In a traditional classroom or staff workshop, students who have already practiced conversational coding often struggle when projects exceed a single file or session. They experience silent requirement drift, hallucinated helper methods, and context-window amnesia. This course trains learners to replace ad-hoc prompting with two complementary open-source engines—**Conductor** for repository context and track execution, and **OpenSpec** for delta proposals, Socratic requirement discovery, and canonical specification merging—culminating in a **Single Source of Truth (`SPEC.md`)** verified by a clean-room rebuild test.

**Backward Design Curriculum Engineering** structures the entire seven-module course around the terminal engineering competency learners must demonstrate in the final capstone [4]. Rather than lecturing abstractly about specification templates and hoping students connect them to code, the curriculum begins with the end state in mind: can a developer hand only an eight-section `SPEC.md` and a read-only test suite to a fresh coding agent in an isolated directory and pass 100% of behavioral and adversarial tests on the first run? Every preceding module and laboratory builds one deliberate layer of that capability, progressing from greenfield Socratic discovery (Lab 01) and brownfield specification recovery (Lab 02) to test-driven actuation (Lab 03), drift detection and five-step scorecard refactoring (Lab 04), specification-backed code review and stacked pull requests (Lab 05), and finally the clean-room rebuild test (Lab 06).

## 1.2 Sweller's Cognitive Load Theory and the Unified `TaskFlow` Progression

**Extraneous Cognitive Load Minimization** is the primary instructional design principle governing the six hands-on laboratories. In cognitive load theory, learners possess a finite working-memory budget that is split between *intrinsic load* (the inherent complexity of the domain), *germane load* (the mental effort spent constructing durable engineering schemas for Conductor and OpenSpec), and *extraneous load* (the friction of re-learning unfamiliar business domains, build systems, or external APIs in every exercise). When a course switches from an e-commerce checkout app in Lab 1 to a cryptocurrency ledger in Lab 2 and a Kubernetes scheduler in Lab 3, students spend 70% of their lab period deciphering domain rules rather than mastering specification engineering.

**The Unified `TaskFlow` Application** eliminates extraneous domain switching by anchoring all six laboratories inside a single, relatable product ecosystem: a personal productivity and Kanban workspace called **TaskFlow**. Each lab isolates one microservice module of `TaskFlow`, bounded to approximately 50 lines of standard-library-only Python 3.10+ and exactly four numbered requirements (`REQ-0001` through `REQ-0004`):

- **Lab 01 (`TodoList` — Greenfield Proposal to Specs):** Students transform a vague three-sentence product idea into an unambiguous OpenSpec delta (`REQ-0001..0004`), two Architecture Decision Records (`ADR-0001` storage engine and `ADR-0002` monotonic ID assignment), and a Conductor execution track.
- **Lab 02 (`TaskOrganizer` — Brownfield Spec Recovery):** Students act as software archaeologists using `/opsx:explore` to extract a ground-truth specification and tag-normalization ADR from undocumented legacy code (`service/task_organizer.py`) without altering runtime behavior.
- **Lab 03 (`HabitTracker` — From Specs to TDD Code):** Students actuate a frozen specification (`specs/spec.md`) into failing-then-passing unit tests (`test_req0001..0004`) and implementation code governing consecutive-day streak arithmetic.
- **Lab 04 (`DailyPlanner` — Drift Detection & 5-Step Refactoring):** Students audit a drifted capacity scheduler (`used_minutes + minutes >= max_minutes`), triage a review `scorecard.md` to reject scope creep, repair the off-by-one boundary bug, and restore behavior-locking tests.
- **Lab 05 (`NotesBoard` — SDD Code Review & Stacked PRs):** Students review a subpar pull request (`subpar_change.patch`) against canonical specs, catch three specification violations (broken ordering, unspecced `pin_note` feature, and silent missing-ID failure), sync `REQ-0004`, and partition `tasks.md` into stacked PR batches.
- **Lab 06 (`TaskFlowBoard` — Capstone SSOT & Clean-Room Rebuild Test):** Students author a complete eight-section `SPEC.md` for a WIP-limited Kanban board and prove zero hidden state by regenerating `taskflow_board.py` in an isolated `/tmp` workspace against read-only adversarial tests.

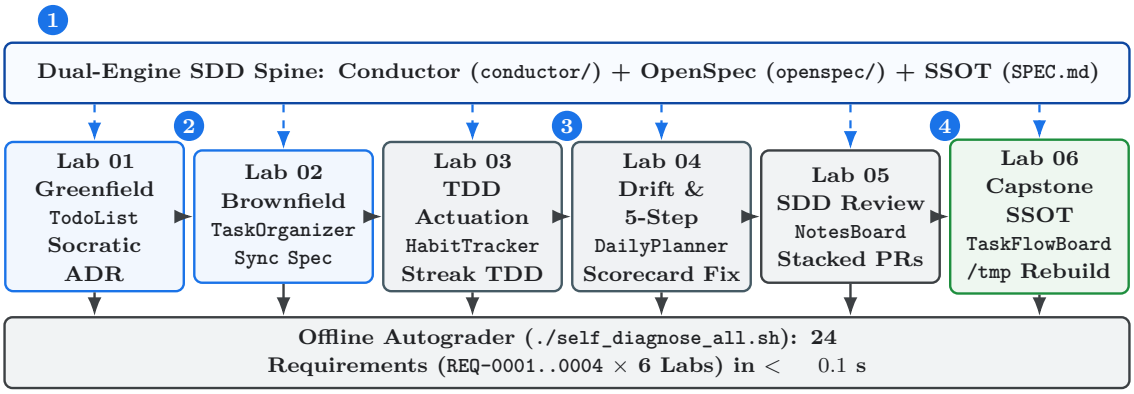



Figure 1: Progressive 6-Lab `TaskFlow` curriculum architecture (Pins [1]–[4]). **Pin [1]** anchors the Dual-Engine SDD Spine (`conductor/` + `openspec/`); **Pin [2]** marks Phase I Specification Authoring & Recovery (Labs 01–02); **Pin [3]** marks Phase II TDD Actuation & Drift Remediation (Labs 03–04); and **Pin [4]** marks Phase III SDD Code Review, Stacked PRs, and Clean-Room SSOT Rebuild Verification (Labs 05–06).

## 1.3 Three Turnkey Pacing Syllabi (University, Bootcamp, and Staff Workshop)

**Modular Pacing Flexibility** enables instructors to deploy the exact same 36-slide deck, student textbook, and six-lab repository across three distinct academic and industrial delivery formats without modifying any lab

code or grading scripts. Table 1 provides the turnkey time allocation for an 8-Week University Seminar, a 2-Day Intensive Practitioner Bootcamp, and a 4-Session Staff Engineering Workshop.

Table 1: Turnkey curriculum pacing across University Seminar, 2-Day Bootcamp, and 4-Session Staff Workshop formats.

| **Module & Slides** | Target Lab | Format A: 8-Week University Seminar (2 hr/wk) | Format B: 2-Day Intensive Bootcamp (14 hr total) | Format C: 4-Session Staff Workshop (4 × 2 hr) |
|---|---|---|---|---|
| **Module 01 (Slides 01–06)** | Dual-Engine Setup & Wiring | **Week 1:** Lecture on Vibe Debt, Spec Triangle, Conductor & OpenSpec wiring; CLI onboarding. | **Day 1 (09:00–10:45):** Foundations lecture + live dual-engine repo initialization demo. | **Session 1 (Part A):** 45-min executive foundations + Spec Triangle walkthrough. |
| **Module 02 (Slides 07–11)** | Lab 01: `TodoList` | **Week 2:** Socratic grilling lecture (45 min) + mentored Lab 01 greenfield specs & ADRs (75 min). | **Day 1 (11:00–13:00):** Live Demo 1 (`/opsx:propose`) + hands-on Lab 01 completion. | **Session 1 (Part B):** 75-min paired Lab 01 Socratic discovery & ADR drafting. |
| **Module 03 (Slides 12–16)** | Lab 02: `TaskOrganizer` | **Week 3:** Brownfield archaeology lecture (45 min) + Lab 02 `/opsx:explore` recovery (75 min). | **Day 1 (14:00–15:45):** Live Demo 2 + Lab 02 brownfield spec recovery & tag ADR. | **Session 2 (Part A):** 60-min brownfield `Sync Spec` lab on legacy code. |
| **Module 04 (Slides 17–21)** | Lab 03: `HabitTracker` | **Week 4:** Spec-to-TDD actuation lecture (45 min) + Lab 03 two-batch streak TDD (75 min). | **Day 1 (16:00–17:45):** Live Demo 3 + Lab 03 `/conductor:implement` TDD cycle. | **Session 2 (Part B):** 60-min Lab 03 streak state-machine TDD actuation. |
| **Module 05 (Slides 22–26)** | Lab 04: `DailyPlanner` | **Week 5:** Drift & 5-Step Refactoring lecture (45 min) + Lab 04 scorecard triage & bug fix (75 min). | **Day 2 (09:00–11:15):** Live Demo 4 + Lab 04 `scorecard.md` triage & boundary fix. | **Session 3 (Part A):** 60-min Lab 04 drift audit & anti-scope-creep triage. |
| **Module 06 (Slides 27–31)** | Lab 05: `NotesBoard` | **Week 6:** SDD Code Review & Stacked PRs (45 min) + Lab 05 patch audit & PR batching (75 min). | **Day 2 (11:30–13:30):** Live Demo 5 + Lab 05 `subpar_change.patch` rejection & sync. | **Session 3 (Part B):** 60-min Lab 05 spec-first PR review & stacked commits. |
| **Module 07 (Slides 32–36)** | Lab 06: `TaskFlowBoard` | **Week 7:** 8-Section `SPEC.md` & Rebuild Test (Lab 06). **Week 8:** Capstone Design Exam & oral defenses. | **Day 2 (14:30–17:30):** Live Demo 6 + Lab 06 clean-room `/tmp` rebuild competition. | **Session 4 (Full 2 hr):** Lab 06 `SPEC.md` authoring & live `/tmp` rebuild verification. |

## 1.4 Dual-Track CLI Support (`main` vs. `claude`) and Deterministic Autograding

**Dual-Track Repository parity** allows learners to complete the entire course using either the Google Antigravity CLI (`agy`, default `main` branch) or Anthropic Claude Code (`claude`, `claude` branch) without diverging in architectural rigor. Both branches ship identical laboratory starter code, identical `verify_lab.py` graders, and identical canonical requirements (`REQ-0001..0004`). The only difference lies in the native agent harness configuration files pre-wired in each workspace: the `main` branch uses `GEMINI.md`, `AGENTS.md`, and `.agents/skills/`, whereas the `claude` branch includes `CLAUDE.md` and `.claude/commands/` slash-command shims.

**Offline Zero-Network Verification (`./self_diagnose_all.sh`)** guarantees that classroom grading never

depends on external API quotas, Wi-Fi stability, or LLM non-determinism. Every lab contains a standalone Python grader (`labs/lab_0X_.../verify_lab.py`) that inspects the learner's Markdown specifications, ADRs, Conductor tracks, and Python implementation using deterministic AST and behavioral assertions. Running `./self_diagnose_all.sh` from the repository root copies each lab into a temporary sandbox, applies the reference solution in `solution/`, executes all unit and adversarial test suites, and verifies 100% pass status across all six labs in under 0.1 seconds.

**Common Misconception vs. Engineering Reality — Teaching Tool Syntax Instead of Specification Semantics**

× **Common Instructor Misconception:** Treating the course as a tutorial on memorizing Conductor and OpenSpec slash-command flags, and grading students solely on whether their Python code passes unit tests.

✓ **Pedagogical & Engineering Reality:** Coding agents can already write 50 lines of Python if given a clear contract; what fails in production is human-to-agent specification precision. Every lab grader (`verify_lab.py`) therefore enforces a **two-stage contract**: Stage 1 audits the Markdown specification artifacts (`REQ-0001..0004`, ADRs, `scorecard.md`, or `SPEC.md`) for explicit invariants and boundary conditions, and Stage 2 verifies that the Python implementation and unit tests faithfully realize those exact invariants.

**Stop & Think — Pre-Class Environment Triage for Mixed Cohort Laptops**

Suppose 30 students arrive at a 2-Day Bootcamp: 18 use `agy`, 10 use `claude`, and 2 have corporate laptops with restricted outbound network access that blocks live LLM API calls. How do you keep all 30 learners synchronized on the exact same laboratory checkpoints?

**Instructor Facilitation Key:**

1. Have the 18 `agy` users stay on `git checkout main` and the 10 `claude` users switch to `git checkout claude`; both tracks share identical lab directories and `verify_lab.py` contracts.
2. Because every lab's Socratic questions, OpenSpec deltas, Conductor tracks, and Python tests are 100% local files validated by `python3 verify_lab.py` with zero network calls, the 2 network-restricted students can act as the "coding agent" themselves (or pair with a neighbor's CLI) while running the exact same local verification gates (`./self_diagnose_all.sh -lab lab_01`).

# 2 Lecture Delivery Playbook: Modules 1 to 3 (Slides 01–16)

## 2.1 Module 01 Delivery (Slides 01–06): Foundations, Spec Triangle & Dual-Engine Wiring

**Slide 01–02 Delivery (Why Vibe Coding Fails at Scale & Specificational Coding)** opens the course by contrasting conversational prompting with specification-driven engineering [1, 2]. Begin by asking the cohort how many have experienced an AI agent fixing Bug B only to re-introduce Bug A from 20 minutes earlier. Explain that chat logs are ephemeral working memory, whereas Git-committed specifications are durable institutional memory. On Slide 02, emphasize the core distinction between three paradigms: *conversational coding* (chatting to mutate source files directly), *ephemeral plan mode* (generating a disposable task list for a single turn), and *specificational coding* (maintaining a versioned `spec.md` contract from which code and tests are compiled). Walk

through the **Spec Triangle** and its three canonical operations: `Apply` (*Act on Spec*), `Sync` (*Sync Spec* from brownfield code without mutating behavior), and `Clean` (*Archive* completed deltas into canonical truth).

**Slide 03–04 Delivery (Conductor Engine & OpenSpec Engine)** introduces the two open-source frameworks that power the course [5, 6]. On Slide 03, show how Conductor (`gemini extensions install https://github.com/gemini-cli-extensions/conductor`) establishes repository-wide baseline context under `conductor/` (`product.md`, `tech-stack.md`, `workflow.md`, `tracks.md`) and executes bounded work units under `conductor/tracks/<track_id>/` via six slash commands (`/conductor:setup`, `newTrack`, `implement`, `status`, `review`, `revert`). On Slide 04, contrast Conductor's task execution plane with OpenSpec's (`@fission-ai/openspec`) **Two-Plane Specification Model**: Plane 1 (`openspec/specs/`) stores current canonical truth (`REQ-XXXX` with normative `MUST`/`SHALL` + `WHEN`/`THEN` scenarios), while Plane 2 (`openspec/changes/<slug>/`) isolates proposed deltas (`ADDED`, `MODIFIED`, `REMOVED`) until `/opsx:archive` merges them.

**Slide 05–06 Delivery (Dual-Engine Wiring & The Unified `TaskFlow` Roadmap)** answers the #1 practitioner question: *"When do I invoke OpenSpec versus Conductor, and how do they avoid conflicting files?"* Use Figure 2 to walk students through the architectural decision tree before running **Live Projector Demo 1**. Point out the three physical bridges on Slide 05: `conductor/index.md` links to `openspec/specs/`, `conductor/workflow.md` requires `/opsx:verify` and `/opsx:archive` before track completion, and each track's `spec.md` cites exact `REQ-XXXX` IDs rather than duplicating prose.

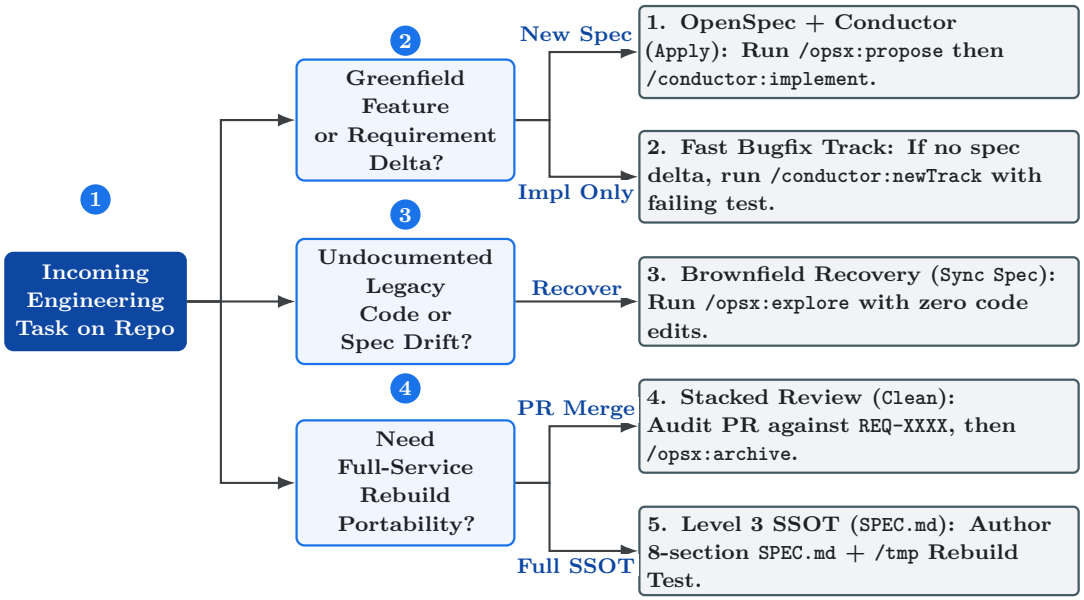



Figure 2: Instructor Decision Tree 1 for routing engineering tasks across OpenSpec and Conductor (Pins [1]–[4]). **Pin [1]** evaluates repository state; **Pin [2]** routes greenfield deltas to `/opsx:propose` + `/conductor:implement`; **Pin [3]** routes undocumented brownfield code to `/opsx:explore` (`Sync Spec`); and **Pin [4]** separates delta archiving (`/opsx:archive`) from full 8-section `SPEC.md` clean-room rebuilds.

## 2.2 Module 02 Delivery (Slides 07–11): Lab 01 — Greenfield Proposals to Specs (`TodoList`)

**Slide 07–09 Delivery (Socratic Grilling & Delta Specification Anatomy)** prepares learners for Lab 01 (`labs/lab_01_greenfield_proposals_to_specs`). Project the raw three-sentence product idea from `product_idea.md`: *"Build an in-memory TodoList in Python where users can add tasks, mark them completed, and delete them."* Ask the room: *"What happens if a user passes `" "` as a title? What if they delete task ID*

*1 and then add a third task—does the new task get ID 3 or collide on ID 2?"* Walk through Slides 08 and 09 showing how Socratic questioning (`Q1..Q3`) eliminates every ambiguity before code is written:

- **`REQ-0001` (Task Creation & Validation):** `add_task(title: str)` `MUST` strip leading/trailing whitespace, assign a monotonically increasing integer ID starting at `1` (`1, 2, 3, ...`), and raise `ValueError` if the trimmed title is empty.
- **`REQ-0002` (Task Completion):** `complete_task(task_id: int)` `MUST` mark the task `completed=True` and return the updated `Task`, or raise `KeyError` if `task_id` does not exist.
- **`REQ-0003` (Task Deletion & ID Monotonicity):** `delete_task(task_id: int)` `MUST` remove the task and return `True` (or return `False` if missing without raising), and `MUST NOT` reuse deleted IDs on subsequent `add_task` calls.
- **`REQ-0004` (Deterministic Listing):** `list_tasks(include_completed: bool = True)` `MUST` return tasks in creation order (ascending `id`).

**Slide 10–11 Delivery (Architecture Decision Records & Conductor Track Handoff)** demonstrates why specifications (*what* the system must do) must be paired with Architecture Decision Records (*why* a specific technical mechanism was selected). Walk through `ADR-0001-storage-engine.md` (selecting an in-memory `dict[int, Task]` over a linear list or SQLite for $O(1)$ lookup and zero external dependencies) and `ADR-0002-id-assignment.md` (selecting a dedicated monotonic counter `self._next_id` over `len(self._tasks) + 1` to prevent ID collision after `delete_task`). Conclude on Slide 11 by showing the 7-section Conductor track (`conductor/tracks/track-01-todo-list/spec.md`) and checkbox task plan (`plan.md`).

## 2.3 Module 03 Delivery (Slides 12–16): Lab 02 — Brownfield Spec Recovery (`TaskOrganizer`)

**Slide 12–14 Delivery (Software Archaeology with `/opsx:explore` and `Sync Spec`)** transitions students from greenfield design to the reality of enterprise engineering: inheriting working, undocumented legacy code (`labs/lab_02_brownfield_spec_recovery/service/task_organizer.py`). Emphasize the cardinal rule of **Brownfield Spec Recovery**: during `/opsx:explore` and `/opsx:sync`, the agent operates in read-only exploration mode over the production code, extracting hidden invariants into `openspec/specs/task-organizer/spec.md` without mutating a single line of `task_organizer.py`.

**Slide 15–16 Delivery (Extracting `TaskOrganizer` Invariants & Tag Normalization ADR)** guides students through the four implicit behaviors hidden inside `service/task_organizer.py`:

1. **`REQ-0001` (Title & Priority Validation):** `add_item(title, priority="medium", tags=None)` strips the title, rejects empty titles with `ValueError`, and validates that `priority` is strictly in `{"high", "medium", "low"}` (raising `ValueError` otherwise).
2. **`REQ-0002` (Write-Time Tag Normalization):** Input tags are stripped, lowercased (`t.strip().lower()`), filtered to remove empty strings, and deduplicated preserving first-seen order into an immutable `tuple[str, ...]`.
3. **`REQ-0003` (Case-Insensitive Tag Filtering):** `filter_by_tag(tag)` normalizes the query with `tag.strip().lower()` and returns matching items in creation order (returning `[]` if the normalized query is empty).
4. **`REQ-0004` (Priority-Ranked Listing):** `list_by_priority()` sorts items by semantic priority rank (`high=0`, `medium=1`, `low=2`) and breaks ties by ascending `id`.

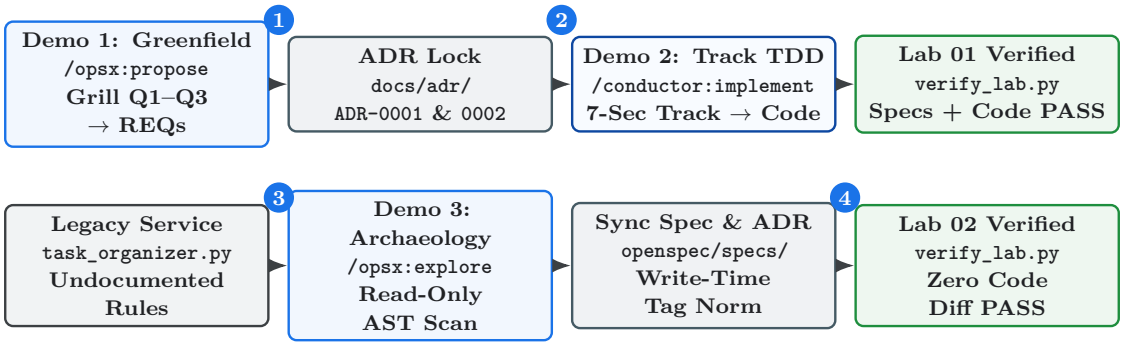



Figure 3: Projector Demo Topology for Modules 1–3 (Pins [1]–[4]). **Pin [1]** shows Greenfield Socratic Discovery locking `ADR-0002` (`_next_id`); **Pin [2]** shows the OpenSpec-to-Conductor handoff in Lab 01; **Pin [3]** shows read-only brownfield exploration (`/opsx:explore`) on `task_organizer.py`; and **Pin [4]** verifies `Sync Spec` completion with zero production code mutation in Lab 02.

**Common Misconception vs. Engineering Reality — Why `len(tasks) + 1` Passes Naive Happy-Path Tests but Fails SDD Review**

× **Common Student Misconception:** Assuming that `new_id = len(self._tasks) + 1` is an acceptable way to assign task IDs in Lab 01 because sequential additions (`add`, `add`, `add`) produce IDs `1`, `2`, `3`.

✓ **Engineering Reality:** If a user adds two tasks (IDs `1` and `2`), deletes task `1` (`len` becomes `1`), and adds a third task, `len(self._tasks) + 1` assigns ID `2` again—overwriting the existing task `2` in `self._tasks[2]`! This is why Socratic Question 3 and `ADR-0002-id-assignment.md` explicitly mandate a monotonic counter (`self._next_id += 1`) that never decrements on deletion.

**Stop & Think — Live Classroom Socratic Challenge — Write-Time vs. Read-Time Tag Normalization**

During Module 03 (Slide 15), pause before showing `ADR-0001-tag-normalization.md` and ask the class: *"In `TaskOrganizer`, why does the legacy code normalize and deduplicate tags at write time inside `add_item()` into a `tuple[str, ...]` rather than normalizing tags at read time inside `filter_by_tag()`?"*

**Instructor Facilitation Key:**

1. **Single Normalization Boundary:** Write-time normalization guarantees that every consumer (`filter_by_tag`, `list_by_priority`, JSON serializers, or UI views) observes clean, lowercase, deduplicated tags (`("urgent", "work")`) without duplicating normalization logic across every read method.
2. **Immutability Contract:** Storing tags as an immutable `tuple[str, ...]` inside `@dataclass(frozen=True) class OrgaTask` prevents callers from mutating `task.tags.append(...)` after creation and violating `REQ-0002`.

# 3 Lecture Delivery Playbook: Modules 4 to 7 (Slides 17–36)

## 3.1 Module 04 Delivery (Slides 17–21): Lab 03 — From Specs to TDD Code (`HabitTracker`)

**Slide 17–19 Delivery (Actuating Specifications via Two-Batch TDD)** introduces Phase II of the course: turning an approved specification (`labs/lab_03_from_specs_to_tdd_code/specs/spec.md`) into deterministic unit tests and production code via `/conductor:implement`. Explain to learners why coding agents fail when asked to implement an entire specification in one shot: without requirement-mapped test batches, the agent blurs boundary conditions and tests its own bugs. On Slide 18, demonstrate how Lab 03 partitions the `HabitTracker` specification into two strict Red-Green-Refactor batches inside `tests/test_habit_tracker.py` and `service/habit_tracker.py`:

- **Batch 1 (`REQ-0001` & `REQ-0004` — Lifecycle & Error Boundaries):** `add_habit(name)` strips whitespace, rejects empty names with `ValueError`, and initializes `streak = 0` and `last_check_in_day = None`; `check_in(name, day)` and `get_streak(name)` raise `KeyError` if `name` is unregistered.
- **Batch 2 (`REQ-0002` & `REQ-0003` — Consecutive Streak State Machine):** First check-in sets `streak = 1`; same-day check-in (`day == last_check_in_day`) is an idempotent no-op returning the current streak; retrograde check-in (`day < last_check_in_day`) raises `ValueError`; consecutive next-day check-in (`day == last_check_in_day + 1`) increments `streak += 1`; and any multi-day gap (`day > last_check_in_day + 1`) resets `streak = 1`.

**Slide 20–21 Delivery (Live Demo 4 — Running `/conductor:implement` on `HabitTracker`)** walks students through executing Batch 1 and Batch 2 on the projector. Point out that each test function in `tests/test_habit_tracker.py` is named after its normative requirement ID (`test_req0001_add_habit`, `test_req0002_idempotent_and_monotonic_day`, `test_req0003_consecutive_vs_gap_reset`, `test_req0004_missing_habit_keyerror`), creating an auditable 1-to-1 trace between `specs/spec.md` and the test runner.

## 3.2 Module 05 Delivery (Slides 22–26): Lab 04 — Drift Detection & 5-Step Scorecard Refactoring (`DailyPlanner`)

**Slide 22–24 Delivery (Diagnosing Spec-to-Code Drift & The 5-Step Refactoring Loop)** prepares students for Lab 04 (`labs/lab_04_drift_detection_and_5step_refactoring`). Project `service/daily_planner.py` alongside `openspec/specs/daily-planner/spec.md` and ask students to spot the subtle off-by-one boundary drift in `REQ-0003`: the canonical spec states that a task of duration `minutes` must be admitted whenever `used_minutes + minutes <= max_minutes`, raising `ValueError` only when `used_minutes + minutes > max_minutes`. However, the drifted code checks `if self.used_minutes + minutes >= self.max_minutes: raise ValueError(...)`, rejecting any task that fills the remaining daily capacity to 100%!

**Slide 25–26 Delivery (Scorecard Triage Against Scope Creep)** teaches learners how to execute the **5-Step SDD Refactoring Playbook** using `scorecard.md`. In Lab 04, an over-eager AI review agent has proposed four findings (`F-01` through `F-04`):

1. **`F-01` (Accepted Defect Fix):** Repair the `>=` boundary operator to `>` in `DailyPlanner.schedule()` so exact-capacity tasks (`used + minutes == max_minutes`) succeed per `REQ-0003`.
2. **`F-02` (Rejected Scope Creep):** Add Google Calendar OAuth synchronization—rejected and recorded in Section 6 (`Out of Scope`) of the track spec.
3. **`F-03` (Rejected Scope Creep):** Add a Pomodoro audio timer thread—rejected and recorded in `Out of Scope`.

4. **`F-04` (Rejected Breaking API Rename):** Rename `schedule()` to `book_slot()`—rejected because it violates the public signature locked in `REQ-0002`/`REQ-0003`.

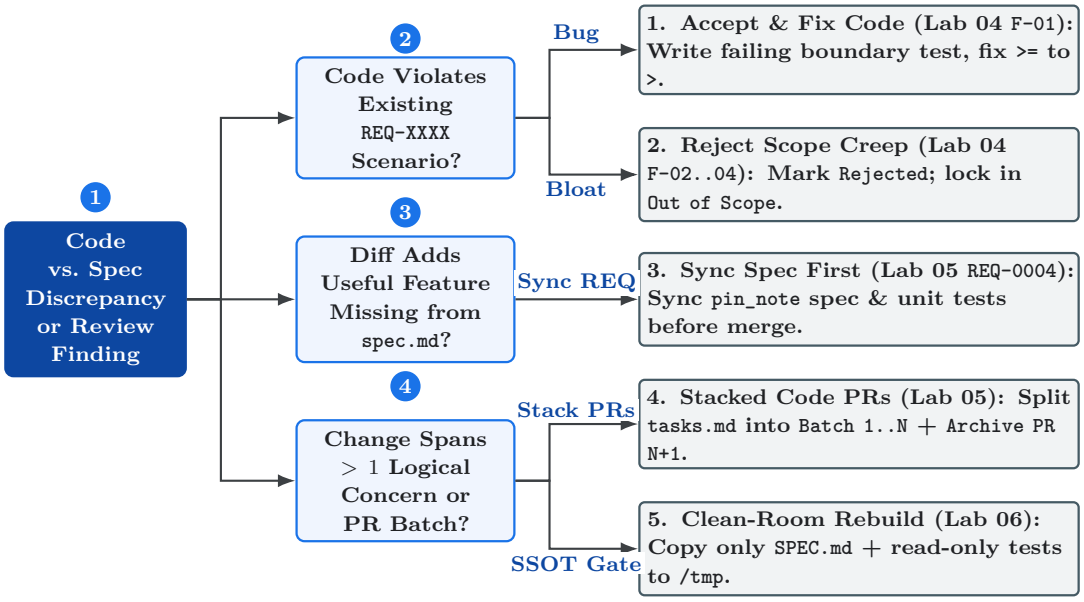



Figure 4: Instructor Decision Tree 2 for triaging spec drift, scope creep, and stacked PR reviews in Modules 5–7 (Pins [1]–[4]). **Pin [1]** classifies the discrepancy; **Pin [2]** separates true spec violations (`F-01`) from scope-creep bloat (`F-02..F-04`); **Pin [3]** enforces `Sync Spec` when a desirable feature (`pin_note`) lacks a `REQ-XXXX` contract; and **Pin [4]** enforces Stacked PR and Clean-Room Rebuild discipline.

## 3.3 Module 06 Delivery (Slides 27–31): Lab 05 — SDD Code Review & Stacked PRs (`NotesBoard`)

**Slide 27–29 Delivery (Spec-Backed Code Review of `subpar_change.patch`)** trains students to review AI-generated pull requests against canonical specifications rather than superficial syntax style (`labs/lab_05_sdd_code_review_and_stacked_prs`). Display `simulated_review/subpar_change.patch` on the projector and guide the class to identify all three specification defects:

1. **Defect 1 (Broken `REQ-0002` Invariant):** The patch wraps `list_notes()` in `reversed(...)`, breaking the canonical requirement that notes `MUST` be returned in creation order (ascending `id`).
2. **Defect 2 (Un-Specced Feature Creep):** The patch introduces `pin_note(note_id)` and `get_pinned_notes()` without adding `REQ-0004` to `openspec/specs/notes-board/spec.md`.
3. **Defect 3 (Silent Error Suppression & Zero Tests):** `pin_note(note_id)` silently returns `None` when `note_id` is missing instead of raising `KeyError` (violating the board's error contract), and ships with zero unit tests.

**Slide 30–31 Delivery (Syncing `REQ-0004` & Stacked PR Partitioning)** shows how to resolve Lab 05 cleanly: students record all three defects in `review_report.md`, synchronize `REQ-0004` (Note Pinning with `KeyError` on unknown ID and creation-order listing in `get_pinned_notes()`) into `openspec/specs/notes-board/spec.md`, repair `service/notes_board.py`, and structure `openspec/changes/add-note-pinning/tasks.md` into **Batch 1 (PR 1: Core Model & `pin_note`)**, **Batch 2 (PR 2: `get_pinned_notes` & Unit Tests)**, and **Dedicated Archive**

**PR N+1 (`/opsx:archive`).**

## 3.4 Module 07 Delivery (Slides 32–36): Lab 06 — Capstone SSOT & Clean-Room Rebuild Test (`TaskFlowBoard`)

**Slide 32–36 Delivery (The 8-Section `SPEC.md` & Clean-Room Rebuild Ceremony)** concludes the lecture series with the Level 3 Spec-as-SSOT capstone (`labs/lab_06_capstone_ssot_and_rebuild_test`) [8]. Walk through the eight mandatory sections of `SPEC.md` (1. System Overview, 2. Domain Data Models, 3. Public Interface & Signatures, 4. Normative Requirements `REQ-0001..0004`, 5. State Machine & WIP Transitions, 6. Error Handling Matrix, 7. Non-Goals / Out of Scope, and 8. Verification & Rebuild Contract). Then run **Live Projector Demo 6**: copy only `SPEC.md` and read-only `adversarial_tests/` (`chmod -R a-w`) into `/tmp/rebuild_check`, synthesize `service/taskflow_board.py`, and watch all unit and adversarial tests pass on the first execution.

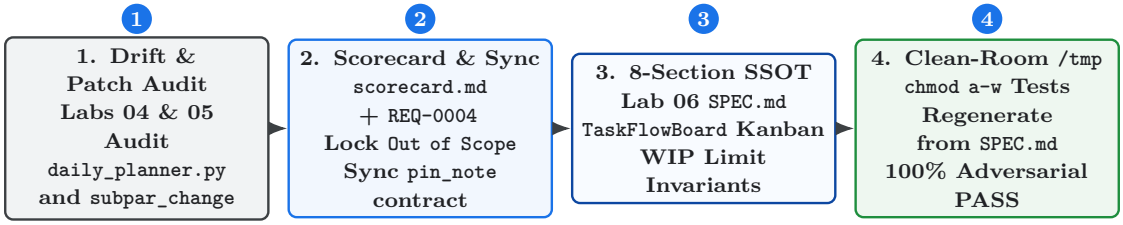



Figure 5: End-to-End Verification Topology across Modules 5–7 (Pins [1]–[4]). **Pin [1]** audits code-to-spec drift and subpar PR diffs; **Pin [2]** enforces scorecard scope-creep rejection and `REQ-0004` synchronization; **Pin [3]** compiles the 8-section `SPEC.md` SSOT for `TaskFlowBoard`; and **Pin [4]** executes the isolated `/tmp` Clean-Room Rebuild Test against read-only adversarial tests.

**Common Misconception vs. Engineering Reality — Running `/opsx:archive` Mid-Stack Instead of in a Dedicated PR `N+1`**

× **Common Student Misconception:** Running `/opsx:archive` inside PR 1 of a 3-PR stacked feature so that the specification delta is merged early.

✓ **Engineering Reality:** Archiving a change moves `openspec/changes/<slug>/` to `archive/` and promotes all `REQ-XXXX` requirements into `openspec/specs/` as live system truth. If PR 1 only implements 40% of the tasks and PR 2 is still in review, archiving in PR 1 causes immediate spec-to-code drift on `main`. Always keep `tasks.md` open across Code PRs $1 \dots N$ and execute `/opsx:archive` in a dedicated, docs-only PR $N+1$ after all code batches are merged.

**Stop & Think — Idempotent Same-Column Moves Under a Full Kanban WIP Limit (Lab 06)**

In Lab 06 (`TaskFlowBoard`), suppose `doing_wip_limit = 2` and two cards (IDs `1` and `2`) are already in the `"doing"` column. A client calls `board.move_card(1, "doing")`. Should this call raise `ValueError("WIP limit reached")` or succeed as an idempotent no-op?

**Instructor Facilitation Key:**

1. **State-Transition Analysis:** Moving card `1` from `"doing"` to `"doing"` does not increase the count of cards in `"doing"` (which remains `2 <= 2`).

2. **Normative Spec Clause (`REQ-0003`):** The adversarial test suite in Lab 06 explicitly tests this edge case! Students must specify in Section 4 (`REQ-0003`) and Section 5 (State Machine) of `SPEC.md` that WIP capacity is checked only when transitioning a card into `"doing"` from a *different* column (`card.column != "doing"`), making same-column moves an idempotent no-op.

# 4 Lab 01–06 Facilitation, Answer Keys & 3-Rung TA Hint Ladders

## 4.1 The 3-Rung Socratic TA Facilitation Protocol

**Graduated Scaffolding Without Giving Away Code** ensures that Teaching Assistants (TAs) help stuck learners discover missing specification clauses or code invariants themselves rather than dictating the solution. Whenever a student raises their hand with a failing `python3 verify_lab.py` check, TAs must escalate strictly through three rungs:

1. **Rung 1 — The Requirement Spotlight (Where to Look):** Point the student to the exact `REQ-XXXX` heading, ADR section, or line range in `verify_lab.py` without naming the fix.
2. **Rung 2 — The $N = 1$ Toy Trace (Concrete Counterexample):** Ask the student to trace a minimal 2-step input sequence by hand (for example: *"Trace what happens when you add 2 tasks, delete task 1, and add a 3rd task"*).
3. **Rung 3 — The Structural Walkthrough (Exact Contract Clause):** Show the exact normative clause or Python guard condition required to satisfy both Stage 1 (Markdown Spec) and Stage 2 (Runtime Code).

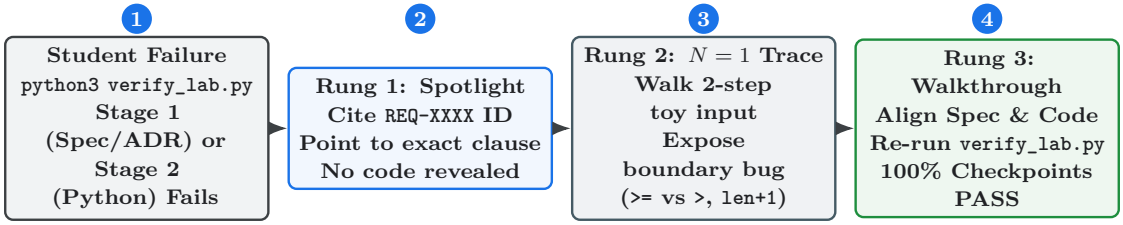



Figure 6: The 3-Rung TA Hint Ladder and Two-Stage Grader Loop (Pins [1]–[4]). **Pin [1]** diagnoses whether failure occurred in Stage 1 (Markdown Spec/ADR) or Stage 2 (Python runtime); **Pin [2]** applies Rung 1 (Requirement Spotlight); **Pin [3]** applies Rung 2 ($N = 1$ counterexample trace); and **Pin [4]** applies Rung 3 to reach 100% verification.

## 4.2 Complete Answer Keys & 3-Rung Hint Ladders for Labs 01–03

**Lab 01 Answer Key (`TodoList` — `labs/lab_01_greenfield_proposals_to_specs`)** verifies five artifacts: `openspec/specs/todo-list/spec.md` (`REQ-0001..0004` with `MUST`/`SHALL` and `#### Scenario:`), `docs/adr/ADR-0001-storage-engine.md` (`Accepted`, choosing `dict[int, Task]`), `docs/adr/ADR-0002-id-assignment.md` (`Accepted`, choosing monotonic `_next_id`), `conductor/tracks/track-01-todo-list/{spec,plan}.md`, and `service/todo_list.py`.

- **Core Python Invariants (`service/todo_list.py`):** `clean = title.strip(); if not clean: raise ValueError(...)`; store `self._tasks: dict[int, Task] = {}` and increment `self._next_id += 1` on every `add_task`; `complete_task` raises `KeyError` if missing; `delete_task` pops and returns `bool` without decrementing `_next_id`.
- **3-Rung Hint Ladder:** *Rung 1:* Check `ADR-0002` and `REQ-0003` on what happens to IDs after deletion. *Rung 2:* Trace `add("A") -> id 1`, `add("B") -> id 2`, `delete(1)`, `add("C")` when using `len(self._tasks) + 1`. *Rung 3:* Replace `len(self._tasks) + 1` with `self._next_id = 1` and increment `self._next_id += 1` only inside `add_task`.

**Lab 02 Answer Key (`TaskOrganizer` — `labs/lab_02_brownfield_spec_recovery`)** requires zero edits to `service/task_organizer.py` and complete recovery of `openspec/specs/task-organizer/spec.md` plus `docs/adr/ADR-0001-tag-normalization.md`.

- **Recovered Spec Clauses:** `REQ-0001` validates non-empty trimmed title and `priority in ("high", "medium", "low")`; `REQ-0002` normalizes tags via `strip().lower()`, drops blanks, and deduplicates preserving first-seen order into a `tuple`; `REQ-0003` filters case-insensitively in creation order; `REQ-0004` orders `list_by_priority()` by `{"high": 0, "medium": 1, "low": 2}` then ascending `id`.
- **3-Rung Hint Ladder:** *Rung 1:* Read lines 28–45 of `service/task_organizer.py` before editing `spec.md`. *Rung 2:* Trace `add_item("Pay bills", "high", ["Urgent ", "urgent", "", "Finance"])`—what is `item.tags`? *Rung 3:* Document write-time `strip().lower()` deduplication into `tuple[str, ...]` in both `REQ-0002` and `ADR-0001-tag-normalization.md`.

**Lab 03 Answer Key (`HabitTracker` — `labs/lab_03_from_specs_to_tdd_code`)** actuates `specs/spec.md` into `service/habit_tracker.py` and `tests/test_habit_tracker.py` (`test_req0001..0004`).

- **Streak State Machine (`check_in(name, day)`):** Raise `KeyError` if `name` not in `self._habits`; raise `ValueError` if `day < 0`; if `last_day is None`, set `streak = 1`; elif `day < last_day`, raise `ValueError("Retrograde day")`; elif `day == last_day`, keep `streak` unchanged; elif `day == last_day + 1`, set `streak += 1`; else (`day > last_day + 1`), reset `streak = 1`. Update `last_check_in_day = day`.
- **3-Rung Hint Ladder:** *Rung 1:* Compare `REQ-0002` (same-day vs. past-day) with `REQ-0003` (next-day vs. multi-day gap). *Rung 2:* Trace `check_in("Read", 5)` followed by `check_in("Read", 5)` and `check_in("Read", 8)`. *Rung 3:* Handle all four branches explicitly (`None`, `< last`, `== last`, `== last + 1`, `> last + 1`) and add a dedicated `test_req000X_*` test for each.

## 4.3 Complete Answer Keys & 3-Rung Hint Ladders for Labs 04–06

**Lab 04 Answer Key (`DailyPlanner` — `labs/lab_04_drift_detection_and_5step_refactoring`)** requires triaging `scorecard.md`, locking `Out of Scope` in `conductor/tracks/track-04-daily-planner/spec.md`, fixing the boundary operator in `service/daily_planner.py`, and adding boundary tests in `tests/test_daily_planner.py`.

- **Core Fix:** In `scorecard.md`, mark `F-01` as `Accepted` and `F-02`, `F-03`, `F-04` as `Rejected`. In `service/daily_planner.py`, change `if self.used_minutes + minutes >= self.max_minutes:` to `if self.used_minutes + minutes > self.max_minutes:`.
- **3-Rung Hint Ladder:** *Rung 1:* Compare `REQ-0003` in `openspec/specs/daily-planner/spec.md` with line 34 of `service/daily_planner.py`. *Rung 2:* If `max_minutes = 60` and you schedule a 60-minute block, what does `0 + 60 >= 60` evaluate to? *Rung 3:* Replace `>=` with `>` and list `F-02..F-04` under Section 6 (`Out of Scope`) of the Conductor track spec.

**Lab 05 Answer Key (`NotesBoard` — `labs/lab_05_sdd_code_review_and_stacked_prs`)** requires documenting the three PR defects in `review_report.md`, syncing `REQ-0004` into `openspec/specs/notes-board/spec.md`, implementing `pin_note` and `get_pinned_notes` in `service/notes_board.py` + `tests/test_notes_board.py`, and batching `openspec/changes/add-note-pinning/tasks.md`.

- **Core Fix:** Keep `list_notes()` in ascending creation order (do not use `reversed()`); in `pin_note(note_id)`, raise `KeyError(f"Unknown note: {note_id}")` if missing; in `get_pinned_notes()`, return `[n for n in self._notes.values() if n.pinned]` in creation order.
- **3-Rung Hint Ladder:** *Rung 1:* Inspect `simulated_review/subpar_change.patch` against `REQ-0002` and `REQ-0003`. *Rung 2:* What does `list_notes()` return after applying the patch if you add Note 1 then Note 2? What does `pin_note(999)` do? *Rung 3:* Document all 3 defects in `review_report.md`, add `REQ-0004` to `spec.md`, raise `KeyError` on missing `note_id`, and split `tasks.md` into `Batch 1 (PR 1)`, `Batch 2 (PR 2)`, and `Archive PR`.

**Lab 06 Answer Key (`TaskFlowBoard` — `labs/lab_06_capstone_ssot_and_rebuild_test`)** requires authoring all 8 sections of `SPEC.md` and implementing `service/taskflow_board.py` + `tests/test_taskflow_board.py` so that a clean-room copy passer in `/tmp` passes `adversarial_tests/test_adversarial_rebuild.py`.

- **Core Kanban Invariants:** Validate `doing_wip_limit >= 1` in `__init__`; `create_card(title)` strips title, rejects blank with `ValueError`, assigns monotonic ID starting at `1` in column `"todo"`; `move_card(card_id, target_column)` validates `target_column in ("todo", "doing", "done")` (`ValueError`) and `card_id in self._cards` (`KeyError`), checks WIP limit *only* when `target_column == "doing" and card.column != "doing"`, and mutates/replaces the card column; `get_column(column)` returns cards in ascending `id` order; `get_wip_summary()` returns `{"todo": ..., "doing": ..., "done": ...}`.
- **3-Rung Hint Ladder:** *Rung 1:* Check Section 5 (State Machine) of `SPEC.md` and `REQ-0003`. *Rung 2:* Trace moving Card 1 from `"doing"` to `"doing"` when `doing_wip_limit == 1`. *Rung 3:* Guard the WIP check with `if target_column == "doing" and card.column != "doing":` and ensure `get_column(col)` validates `col in VALID_COLUMNS`.

**Common Misconception vs. Engineering Reality — Letting Students Patch `verify_lab.py` or `adversarial_tests/`**

× **Common Classroom Pitfall:** Allowing a student (or an unconstrained coding agent running in auto-approve mode) to weaken assertions inside `verify_lab.py` or `adversarial_tests/` so the terminal prints `PASS`.

✓ **Engineering Reality:** This is the exact "Self-Testing Trap" illustrated on Slide 01! In Lab 06, always instruct students to run `chmod -R a-w adversarial_tests/` before invoking their coding agent, and run `git status` / `git diff **/verify_lab.py` during TA sign-off to verify zero grader tampering.

**Stop & Think — Debugging a Silent Stage 1 Specification Failure When Stage 2 Python Tests Pass**

A student in Lab 01 shows you that `python3 -m unittest discover -s tests` passes 100%, yet `python3 verify_lab.py` exits with a Stage 1 failure on `openspec/specs/todo-list/spec.md`. Why did this happen, and what pedagogical lesson should the TA reinforce?

**Instructor Facilitation Key:**

Table 2: Master TA Troubleshooting Matrix for common student failures across Labs 01–06.

| Lab ID | Symptom / Grader Error | Root Cause in Student Workspace | Exact Remediation Command / Fix |
|---|---|---|---|
| **Lab 01** `TodoList` | `AssertionError: expected id 3 after delete, got 2` | Student used `len(self._tasks) + 1` instead of monotonic `self._next_id`. | Add `self._next_id = 1` in `__init__` and increment `self._next_id += 1` in `add_task()`. |
| **Lab 02** `TaskOrganizer` | `FAIL: service/task_organizer.py was modified` | Student edited legacy code instead of running read-only `Sync Spec`. | Run `git checkout - labs/lab_02_.../service/task_organizer.py` and edit only `spec.md` & `ADR-0001`. |
| **Lab 03** `HabitTracker` | `FAIL: Missing test_req0001..0004 in test_habit_tracker.py` | Student wrote generic test names (`test_add`) instead of requirement-mapped names. | Rename unit tests to include `req0001`, `req0002`, `req0003`, and `req0004`. |
| **Lab 04** `DailyPlanner` | `FAIL: scorecard.md did not reject scope creep F-02..F-04` | Student accepted Calendar OAuth or Pomodoro timer in `scorecard.md`. | Mark `F-01 Accepted` and `F-02..F-04 Rejected`; list them in Section 6 `Out of Scope`. |
| **Lab 05** `NotesBoard` | `AssertionError: list_notes() returned reverse order` | Student applied `subpar_change.patch` directly without reverting `reversed()`. | Preserve insertion order in `list_notes()` and raise `KeyError` in `pin_note()` on missing ID. |
| **Lab 06** `TaskFlowBoard` | `ValueError on idempotent move_card(1, "doing")` | WIP check triggered even when card was already in `"doing"`. | Check capacity only when `target_column == "doing" and card.column != "doing"`. |

1. **Root Cause:** The student jumped straight to coding `service/todo_list.py` (Vibe Coding habit!) and left placeholder `TODO` markers or non-normative prose in `openspec/specs/todo-list/spec.md`.
2. **Pedagogical Lesson:** In SDD, passing code with an incomplete specification is an unshippable artifact because the next engineer or agent session will read `openspec/specs/` as system truth. Walk the student through completing `REQ-0001..0004` with explicit `MUST`/`SHALL` and `#### Scenario:` blocks first.

# 5 Assessments, Capstone Grading Rubrics & Primary Literature

## 5.1 100-Point Capstone Specification & Clean-Room Rebuild Grading Rubric

**Dual-Artifact Assessment Architecture** evaluates students on both the precision of their human-authored specification artifacts (55 points) and the zero-shot clean-room rebuild fidelity of their resulting service (45 points). Table 3 provides the standardized 100-point grading rubric for university term projects and staff engineering certification cohorts.

Table 3: 100-Point Grading Rubric for the SDD Capstone (`SPEC.md` + Dual-Engine Repo + Clean-Room Rebuild Test).

| Assessment Dimension | Weight | Exemplary (90–100% of pts) | Proficient (70–89% of pts) | Developing (< 70% of pts) |
|---|---|---|---|---|
| **1. Normative Requirements & Scenarios (`REQ-XXXX`)** | 25 pts | All requirements use RFC 2119 `MUST`/`SHALL` keywords and testable `WHEN`/`THEN` scenarios covering happy, boundary, and error paths. | Requirements are numbered and clear, but 1–2 boundary or tie-breaking cases are left implicit. | Vague user stories without normative keywords or verifiable `WHEN`/`THEN` acceptance criteria. |
| **2. Architectural Decision Records (`docs/adr/`)** | 15 pts | Every non-trivial data structure, ID invariant, or normalization rule has an `Accepted` ADR comparing $\geq 3$ options with explicit trade-offs. | ADRs exist and state decisions, but omit rejected alternatives or failure modes of naive options. | Missing ADRs, or ADRs that merely restate what the code does without architectural trade-off analysis. |
| **3. Dual-Engine Hygiene & Anti-Scope-Creep** | 15 pts | Clean separation between `openspec/specs/` and `conductor/tracks/`; explicit `Out of Scope` section and stacked PR batching in `tasks.md`. | Both engines initialized and linked, but minor duplication between track spec and canonical spec. | Un-specced features merged directly to code, or `/opsx:archive` run prematurely mid-stack. |
| **4. Requirement-Mapped TDD Suite** | 20 pts | 1-to-1 unit test functions (`test_req0001..`) verifying every normative scenario, boundary condition, and exception type (`ValueError`/`KeyError`). | Unit tests pass and cover main paths, but miss 1 off-by-one boundary or exception check. | Tests only cover happy paths, or test assertions were weakened to match buggy code. |
| **5. Clean-Room `/tmp` Rebuild Test** | 25 pts | Copying *only* `SPEC.md` + `chmod a-w` adversarial tests to `/tmp` regenerates a 100% passing service on the first run with zero manual code edits. | Clean-room rebuild passes $> 85\%$ of adversarial tests; requires 1 minor spec clarification (fixed in `SPEC.md`, not code). | Rebuild fails multiple adversarial tests, or student manually edits generated code in `/tmp` instead of fixing `SPEC.md`. |

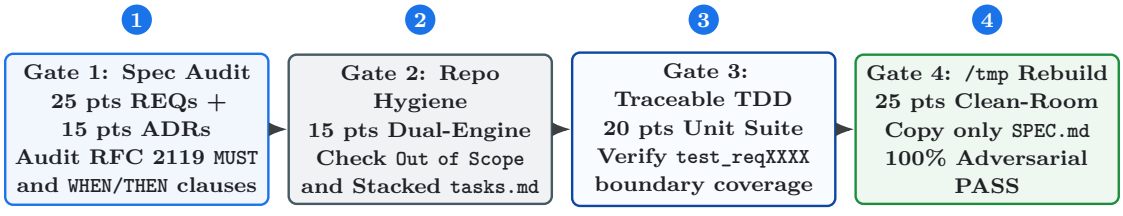



Figure 7: The 100-Point Capstone Assessment Pipeline (Pins [1]–[4]). **Pin [1]** grades normative requirements and ADRs (40 pts); **Pin [2]** audits dual-engine wiring and scope-creep barriers (15 pts); **Pin [3]** checks requirement-mapped TDD coverage (20 pts); and **Pin [4]** executes the Clean-Room `/tmp` Rebuild Test (25 pts).

## 5.2 Six Open-Ended Architectural Design Exam Problems & Model Grading Keys

**Scenario-Based Architectural Evaluation** replaces rote multiple-choice quizzes with six open-ended design and verification problems that test whether learners can diagnose specification failures and author unambiguous contracts.

1. **Problem 1 — Diagnosing the Ephemeral Plan Mode Trap:** An engineering team uses their coding agent's built-in "Plan Mode" for every pull request, yet after six weeks their microservice suffers from severe architectural drift across modules. Explain why ephemeral plan mode fails to prevent cross-session drift and specify the exact two-plane file structure required to fix it.
   - *Model Grading Key (15 pts):* Ephemeral plans live only in chat history for a single turn and vanish after commit (5 pts). Fixing this requires Plane 1 canonical specs (`openspec/specs/<capability>/spec.md`) + persistent repository context (`conductor/product.md`, `tech-stack.md`) and Plane 2 delta staging (`openspec/changes/<slug>/`) merged via `/opsx:archive` (10 pts).
2. **Problem 2 — Socratic Grilling & Monotonic Identity Contracts:** Given the prompt *"Add a comment thread service where users can post comments and moderators can delete spam comments,"* write three Socratic questions and draft `ADR-0001` preventing ID reuse bugs.
   - *Model Grading Key (15 pts):* Questions must cover blank/whitespace body validation, missing-ID behavior on delete (`KeyError` vs. `bool`), and ordering of remaining comments (8 pts). `ADR-0001` must reject `len(comments) + 1` in favor of a monotonic counter `_next_id` so deleting comment $k$ never causes a subsequent comment to collide with an existing ID (7 pts).
3. **Problem 3 — Brownfield Archaeology (`Sync Spec` vs. Refactoring):** During an `/opsx:explore` audit of a legacy billing module, an engineer notices that currency codes are normalized via `code.strip().upper()` at write time, but also spots an opportunity to rewrite the storage dict into SQLite. Should the engineer perform both in the same PR?
   - *Model Grading Key (15 pts):* Strictly no (5 pts). Brownfield `Sync Spec` (`/opsx:explore` → `/opsx:sync`) must extract current ground-truth behavior into `openspec/specs/` with *zero* production code changes first so regression tests lock existing behavior; only after the baseline spec is committed should a separate delta proposal (`/opsx:propose`) propose a storage migration (10 pts).
4. **Problem 4 — Off-by-One Drift & 5-Step Scorecard Triage:** A rate-limiter specification states that a client may make up to `max_requests` per window, rejecting requests only when the new total exceeds `max_requests`. The implementation writes `if current_count + 1 >= max_requests: raise RateLimitError`. Trace the boundary failure for `max_requests = 1` and explain how to remediate it using the 5-Step Refactoring Playbook.
   - *Model Grading Key (15 pts):* At `max_requests = 1` and `current_count = 0`, the very first request evaluates `0 + 1 >= 1` as `True` and raises `RateLimitError`, reducing effective capacity to zero (7 pts). Remediation: (1) audit against spec, (2) accept boundary defect in `scorecard.md` while rejecting unrelated feature suggestions into `Out of Scope`, (3) write a failing boundary test at exact capacity, (4) change `>=` to `>`, and (5) verify and checkpoint (8 pts).
5. **Problem 5 — Stacked PR Partitioning & Archive Sequencing:** An OpenSpec change `add-webhook-retries` requires 9 tasks across 3 subsystems (schema, worker, CLI) plus archiving the delta. Design the Git/Jujutsu stacked PR sequence and explain why `/opsx:archive` must be isolated in the final PR.

- *Model Grading Key (20 pts):* Partition `tasks.md` into Batch 1 (PR 1: Schema + unit tests), Batch 2 (PR 2: Retry worker + unit tests), Batch 3 (PR 3: CLI flags + integration tests), and PR 4 (Dedicated docs-only `/opsx:archive` PR) (10 pts). Archiving earlier promotes unimplemented requirements to `openspec/specs/` on `main`; isolating `/opsx:archive` in PR 4 guarantees `openspec/specs/` matches merged code at every commit (10 pts).

6. **Problem 6 — Designing a Clean-Room Rebuild Test:** Specify the exact shell commands and directory permissions required to run a valid Clean-Room Rebuild Test on a service, and explain how to respond if the rebuilt code fails 1 of 12 adversarial tests.

   - *Model Grading Key (20 pts):* Create an empty directory in `/tmp`, copy *only* `SPEC.md` and `adversarial_tests/`, lock tests read-only with `chmod -R a-w adversarial_tests/`, and instruct a fresh agent session to synthesize the implementation from `SPEC.md` alone (10 pts). If 1 test fails, never patch the generated code in `/tmp` by hand; instead, identify the underspecified invariant in `SPEC.md`, strengthen `SPEC.md`, and re-run the clean-room rebuild from scratch (10 pts).

**Common Misconception vs. Engineering Reality — Fixing Clean-Room Rebuild Failures by Editing the Generated `/tmp` Code**

× **Common Student Misconception:** When an adversarial test fails during the Lab 06 Clean-Room Rebuild Test in `/tmp`, manually editing the generated Python file inside `/tmp` until the test passes.

✓ **Engineering Reality:** A Clean-Room Rebuild Test is a test of **`SPEC.md`**, not a test of manual coding! If a fresh coding agent misinterprets an edge case (such as idempotent same-column Kanban moves), it proves that `SPEC.md` is ambiguous. The only valid fix is to update `SPEC.md` in the source repository and re-run the clean-room generation from zero.

**Stop & Think — Oral Defense Question for Capstone Sign-Off**

During a 5-minute capstone oral defense, ask the student to delete their `service/taskflow_board.py` file live in front of you and regenerate it from their `SPEC.md`. Why does this single exercise transform student attitudes toward documentation?

**Instructor Facilitation Key:**

1. **Inverting the Value Hierarchy:** Students who view documentation as a chore treat source code as precious and specifications as disposable commentary.
2. **Experiencing Spec-as-Source:** Watching `service/taskflow_board.py` get deleted and cleanly re-compiled from `SPEC.md` in 15 seconds proves viscerally that the specification is the true source code and the Python file is merely a compiled target.

## 5.3 Instructor's Annotated Primary-Source Literature Guide

**Curated Primary Sources** ground every lecture module in peer-reviewed software engineering research, empirical industry analyses, and official open-source framework documentation. Assign readings [1, 2, 3] prior to Module 01, [5, 6] alongside Modules 02–05, and [4, 7, 8] ahead of the Module 07 capstone.

# References

[1] A. Karpathy, "Vibe Coding and LLM-Assisted Software Engineering Workflows," Technical Commentary & Practitioner Notes, 2025. *Instructor Note: Use in Module 01 (Slide 01) to contrast unconstrained conversational prompting with Karpathy's disciplined context-scoping and small-diff verification rules.*

[2] Thoughtworks Technology Radar, "Spec-Driven Development (SDD) with AI Coding Agents," Thoughtworks, 2025. *Instructor Note: Primary reference for Module 01 (Slide 02) defining why versioned specifications replace ephemeral chat history in multi-session engineering.*

[3] B. Böckeler, "Exploring Spec-Driven Development: Spec-First, Spec-Anchored, and Spec-as-Source," Martin-Fowler.com, 2025. *Instructor Note: Essential taxonomy for Table 1 in the Student Coursebook and Slide 02 (The 3 Maturity Levels of SDD).*

[4] M. Fowler, "Context Engineering and Specificational Coding for AI-Assisted Delivery," MartinFowler.com, 2025. *Instructor Note: Background reading for Module 01 and Module 05 on how curated repository context and living specifications prevent context rot.*

[5] Google Gemini CLI Extensions, "Conductor: Context-Driven Development Extension (`https://github.com/gemini-cli-extensions/conductor`)," Apache-2.0 Open-Source Repository, 2025. *Instructor Note: Primary technical reference for Module 01 (Slide 03), Lab 01, Lab 03, and Lab 04 track workflows.*

[6] Fission AI, "OpenSpec: Two-Plane Specification Engine for AI Coding Agents (`https://github.com/Fission-AI/OpenSpec`)," MIT Open-Source Repository, 2025. *Instructor Note: Primary technical reference for Module 01 (Slide 04), Lab 01 (`/opsx:propose`), Lab 02 (`/opsx:explore`), and Lab 05 (`/opsx:archive`).*

[7] J. Sweller, "Cognitive Load During Problem Solving: Effects on Learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988. *Instructor Note: Theoretical foundation for Section 1.2 of this Lecturer Manual explaining why all six labs use the unified `TaskFlow` domain.*

[8] D. L. Parnas, "On the Criteria To Be Used in Decomposing Systems into Modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972. *Instructor Note: Classic software engineering foundation for Module 02 ADRs and Module 07 information-hiding interface contracts in `SPEC.md`.*